

Minutes of Meeting (MoM)

Topic: Building High-Quality AI Agents

Conducted by: Ankur Sir

Participants: BA Team & Development Team

Date: 14 April 2026

1. Objective of the Meeting

To define:

- How to build **high-quality AI agents**
 - Standard **architecture, behavior, and lifecycle**
 - **Company-wide rules and best practices** for all AI initiatives
-

2. Understanding AI Agent Architecture

An AI agent consists of the following core components:

- **LLM (Large Language Model)** – Executes instructions
- **Prompt Layer** – Converts intent into instructions
- **Tools & Skills** – Enable execution capabilities
- **Context & Memory Engine** – Maintains relevant information
- **Execution Layer** – Handles task completion

Agent Flow

User Input → Intent → Instruction → LLM Execution → Tool/Skill Usage → Output

3. Agent Lifecycle

Agents can operate in:

- **Sequential mode**
- **Parallel mode**
- **Remote execution environments**

Lifecycle Stages

1. Input / Intent Capture
2. Intent Evaluation (Sufficiency Check)
3. Instruction Generation
4. Execution via LLM
5. Tool/Skill Invocation
6. Output Generation
7. Logging & Learning

Intervention can happen at **any stage of lifecycle** for optimization.

4. Characteristics of a Good AI Agent

A high-quality agent must:

- Be **proactive and curious**
 - Be **intelligent at all times**
 - **Understand when to say less and when to say more**
 - Give **sharp and precise answers**
 - Ask **minimum but relevant questions**
 - Be **grounded in context**
 - Complete tasks in **minimum number of steps**
 - **Remember the right information in the right way**
 - Never behave in a **confused or “dumb” manner**
-

5. Core Capabilities of Every Agent

Every agent must have the following core skills:

- **Read**
- **Write**
- **Think**
- **Act**

These should be implemented as **reusable skill modules**.

6. Tools, Skills & Libraries (Standardization)

Company Rule: Common Libraries

- All AI initiatives must use:
 - **Common Tool Library**
 - **Common Skill Library**
 - **Common CLI Interface**

Key Files

- `skills.md` → Skill definitions
- `agent.md` → Agent documentation
- `README.md` → Full project understanding

LLMs understand **.md format naturally**, so documentation must be in Markdown.

7. Prompt Engineering Guidelines

- Prompts must be **fragmented (modular)**, not combined
- Each part of the prompt should serve a **clear purpose**
- Prompts = **Well-defined instructions**, NOT user text

Important Concept

- User input is **NOT** a prompt → it is an **INTENT**

Flow

Intent → Instruction → Execution

8. Intent Handling & Decision Logic

Intent Sufficiency Check

1. If intent is **sufficient** → **proceed**

2. If **intent + system knowledge is sufficient** → **proceed**
 3. If **insufficient** → **ask minimal clarifying questions**
-

9. Instruction Design

- Instructions must be:
 - **Clear**
 - **Structured**
 - **Non-emotional**
 - **Actionable**

Slot Filling Approach

- What user has
 - What is missing
 - What is required
-

10. Prompt Optimization

- Use tools like **DSPy** for prompt optimization
 - Continuously refine prompts for:
 - Accuracy
 - Efficiency
 - Clarity
-

11. Zero Hardcoding Policy

- **Strict Rule:** No hardcoding in core logic
- Allowed ONLY when:
 - **100% deterministic**
 - Defined as **explicit rules**

Rules must be **separately defined**, not embedded in core code.

12. Documentation & Design Standards

Before building any agent:

Mandatory

- Create **Mermaid Diagram** including:
 - Agent flow
 - Sub-agents
 - Tools
 - Skills
 - CLI interactions
- Maintain:
 - `agent.md`
 - `skills.md`
 - Proper repository documentation

Any developer should understand the system **just by reading the repo**

13. Context & Memory Management

- Use a **Memory Engine** (e.g., Google Memory tools)
- Maintain **context selectively**
- Be **choosy about what to store and retrieve**

Rule

A good agent remembers the **right information in the right manner**

14. Logging & Observability

- All interactions must be:
 - **Logged**
 - **Tracked (tokens, instructions, outputs)**

Ensures debugging, optimization, and transparency.

15. Learning & Self-Evolving Agents

- Agents should:
 - Learn from **feedback loops**
 - Improve over time

Concepts

- **Self-evolving agents (Mem0 / Memento-like systems)**
 - Continuous improvement via:
 - User interactions
 - Memory updates
-

16. Avoiding Agent Failures

Common Issues:

- Hallucinations
- Poor prompt structure
- Language barriers
- Context overload

Solutions:

- Clear prompt fragmentation
 - Strong instruction design
 - Controlled context usage
-

17. Research & Continuous Learning

Teams must actively track:

- **Google Scholar** (latest AI research)
 - **GitHub Trending Repositories**
 - Top contributors and innovations
-

18. Key Takeaways (Company Rules)

1. Build **intelligent, proactive agents**

2. Follow **modular prompt architecture**
 3. Maintain **common libraries** (tools, skills, CLI)
 4. Enforce **zero hardcoding policy**
 5. Always create **design diagrams before coding**
 6. Treat user input as **intent, not prompt**
 7. Implement **strong memory & context management**
 8. Ensure **full logging & observability**
 9. Build **self-evolving systems**
 10. Maintain **clear documentation in Markdown**
-

19. Action Items

Define common tool & skill libraries

Create standard agent templates (`agent.md`,
`skills.md`)

Implement logging & memory engine

Setup prompt optimization framework (DSPy)

Prepare Mermaid diagrams before development

Research latest AI advancements

Conclusion:

The company will follow a **structured, modular, and scalable approach** to building AI agents, focusing on **intelligence, efficiency, standardization, and continuous learning**.